

5.3 WiFi Stability ESP32

ESP32 WiFi Stability (with Web Configuration Portal)



Note: This tutorial builds on the previous two parts (STA mode, AP mode) and explains how to keep your ESP32's WiFi connection **stable over the long term**. The focus is on automatically starting AP mode and setting up a **Web Configuration Portal** when STA connection fails, allowing users to modify STA credentials via a phone/computer browser and reboot ESP32 with one click for self-recovery. Please complete the STA and AP mode tutorials first.

Why Do We Need "Stability Maintenance"?

In the previous two projects, you've learned how to make ESP32 connect to a router (STA) or become a hotspot (AP). But in real-world scenarios, WiFi is not always "smooth sailing":

- ▶ Router occasionally reboots → ESP32 disconnects
- ▶ Signal weakened by walls → intermittent connection
- ▶ **Password changed → can never connect again** (most common deployment issue)
- ▶ DHCP timeout → no IP obtained
- ▶ ESP32 itself has issues → freezes

In **demo scenarios**, when disconnected, you just press the reset button, or modify code and re-upload. But in **actual deployment** scenarios, the device may be installed in ceilings, outdoor electrical boxes, or remote cabinets, making on-site operation impossible.

The most typical pain point: When a user changes their home router's WiFi password, the ESP32 can no longer connect. To make the device work again, the traditional approach is: remove the device → connect to computer → modify code → re-upload. This is almost infeasible for large-scale deployment.

This tutorial's solution: When ESP32 cannot connect, it automatically becomes a WiFi hotspot and starts a small web server. The user connects to this hotspot with their phone, opens [192.168.4.1](#) in the browser, directly enters the new WiFi credentials on the webpage, clicks "Save and Reboot", and ESP32 will reconnect with the new password. **No computer needed, no coding needed, no program upload needed.**

Related Knowledge

1. Common Causes of WiFi Instability

Cause	Symptom	This Solution's Response
Router reboot	Brief disconnect, auto-recovers	Exponential backoff reconnection, auto-recovery
Weak signal	RSSI < -80 dBm, frequent disconnects	Reconnection + statistics alerting
Password changed	Can never connect	Web portal to modify password
DHCP failure	Associated but no IP obtained	Timeout then enter AP fallback
ESP32 abnormality	Program freeze or memory leak	Watchdog hard reboot

2. STA and AP Mode Review

Mode	Role	Role in Stability Solution
STA	Client connecting to router	Primary working mode, tried first
AP	Acts as hotspot itself	Fallback mode, activated when STA fails, sets up Web Configuration Portal



Core idea of this project: Prefer STA mode to connect to router; if multiple failures occur, automatically switch to AP mode, start the Web Configuration Portal, let user modify STA credentials via browser and reboot with one click, ESP32 reconnects with new credentials.

3. What is a Web Configuration Portal?

A **Web Configuration Portal** means ESP32 runs a small HTTP server in AP mode. Users access its IP address via browser, see a web form, enter new WiFi credentials in the form, and after submission, ESP32 saves the credentials to Non-Volatile Storage (NVS), then reboots and connects to the router with the new credentials.

4. What is NVS (Non-Volatile Storage)?

NVS (Non-Volatile Storage) is a special flash storage area on ESP32 where data **is not lost** after power off. The Arduino framework provides access through the [Preferences](#) library.

Simple analogy: NVS is like ESP32's "little notebook" that can still be read after reboot. WiFi credentials are recorded in this notebook, so they can be read every time it boots, without needing to reconfigure each time.

NVS characteristics:

- Not lost on power off (like flash)
- Limited read/write cycles (about 100,000 times, sufficient)
- Organized by namespace, similar to folders
- Key-value pair storage, similar to a dictionary

This project uses namespace [wifi_creds](#) and stores two keys:

- [ssid](#) : WiFi name
- [pass](#) : WiFi password

5. Two Programming Paradigms for ESP32 WiFi

(1) Polling

Repeatedly call [WiFi.status\(\)](#) in [loop\(\)](#) to query status. This is the approach used in the previous two projects, simple and intuitive.

(2) Event-Driven

Register a callback function that ESP32 firmware **automatically calls** when WiFi status changes.

```
1 | WiFi.onEvent(onWiFiEvent); // Register callback
2 |
3 | void onWiFiEvent(WiFiEvent_t event) {
4 |   if (event == ARDUINO_EVENT_WIFI_STA_GOT_IP) {
5 |     // Got IP, can start business logic
6 |   }
7 | }
```

Advantages: Timely response, decoupled code, no CPU waste.



This project uses **Event-Driven + State Machine + Web Portal** combination, which is the mainstream approach for industrial-grade WiFi applications.

6. State Machine Concept

A **State Machine** is a programming model that breaks complex processes into "states + transitions". The system is in only one state at any time, and jumps to another state when specific events occur.

This project defines 6 states:

State	Meaning	LED Behavior	Web Portal
BOOT	Just powered on	Off	-
STA_TRYING	Trying STA connection	Slow blink (500ms)	-
STA_OK	STA connected, working normally	Solid on	-
STA_LOST	STA disconnected, retrying	Fast blink (150ms)	-
AP_FALLBACK	STA failed, fell back to AP	Double-blink	Running
FAIL	AP also failed, waiting for watchdog reboot	Off	-

7. Key Stability Technologies Overview

Technology	Purpose	Implementation to Elegoo
Event-driven callback	Real-time perception of WiFi status changes	<code>WiFi.onEvent()</code>
Non-blocking reconnect	Doesn't block main loop, can do other things	<code>millis()</code> timing
Exponential backoff	Avoid overwhelming router with frequent reconnects	<code>reconnectInterval *= 2</code>
AP fallback + Web portal	Let user modify password via webpage when STA fails	<code>startAPFallback()</code> + <code>WebServer</code>
NVS persistence	Credentials survive power off, auto-read on reboot	<code>Preferences</code> library
One-click reboot	Remote reboot ESP32 via webpage button	<code>ESP.restart()</code>
Watchdog reboot	Hard reboot in extreme cases, self-healing	<code>handleFail()</code>
LED status indicator	See current status without computer	<code>updateLED()</code>
State statistics	Record reconnect counts etc. for diagnostics	<code>reconnectAttempts</code> etc.

Library Installation Guide

This project uses three libraries bundled with ESP32 board support, no additional installation needed:

Library	Purpose
<code>WiFi.h</code>	WiFi connection, AP mode, event callbacks
<code>WebServer.h</code>	HTTP server, provides web configuration portal
<code>Preferences.h</code>	NVS non-volatile storage, saves WiFi credentials

If you haven't installed ESP32 board support yet, please refer to the "Library Installation Guide" in Part 1 "WiFi Connection Tutorial".

Key Functions

1. WiFi Event-Related Functions

Function	Description
<code>WiFi.onEvent(callback)</code>	Register WiFi event callback function
<code>WiFi.setAutoReconnect(bool)</code>	Enable/disable underlying auto-reconnect (set to <code>false</code> in this project, manual management)

2. Common WiFi Events

Event Constant	Trigger Timing
<code>ARDUINO_EVENT_WIFI_STA_START</code>	STA mode started
<code>ARDUINO_EVENT_WIFI_STA_CONNECTED</code>	STA associated with router (but no IP yet)
<code>ARDUINO_EVENT_WIFI_STA_GOT_IP</code>	STA obtained IP address (truly connected)
<code>ARDUINO_EVENT_WIFI_STA_DISCONNECTED</code>	STA disconnected
<code>ARDUINO_EVENT_WIFI_AP_START</code>	AP mode started



Key point: Only when `GOT_IP` event is received is it truly connected. `CONNECTED` only means associated with router, may still be waiting for DHCP.

3. WebServer Library Core Functions

[Back to Elegoo](#)

Function	Description
WebServer server(port)	Create HTTP server object, specify port
server.on(path, method, handler)	Register route: path + method (GET/POST) + handler function
server.begin()	Start server, begin listening
server.handleClient()	Handle client requests (must be called repeatedly in loop)
server.send(code, type, content)	Send HTTP response
server.arg("name")	Get form-submitted field value
server.stop() / server.close()	Stop server
server.onNotFound(handler)	Register 404 handler function

4. Preferences Library (NVS) Core Functions

Function	Description
prefs.begin("namespace", readOnly)	Open namespace, true read-only, false read-write
prefs.putString("key", value)	Store string
prefs.getString("key", default)	Read string, returns default if not exists
prefs.end()	Close namespace (must be called, otherwise data may not flush)

5. Reconnection and Diagnostic Functions

Function	Description
WiFi.disconnect()	Disconnect current connection (call before reconnect)
WiFi.RSSI()	Get signal strength (dBm)
WiFi.softAPgetStationNum()	Get connected client count in AP mode
ESP.restart()	Soft reboot ESP32

Complete Code Analysis (By Module)

 [DOWNLOAD](#)

Reading Guide: This code is divided into **10 modules** by function. Each module is responsible for one independent category of functionality, with code and explanation together. Recommended to read in order, but modules are relatively independent and can be skipped as needed.

Module dependency diagram:

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- 10
- Module 1 (Config) —————> All modules reference parameters here
- Module 2 (State Machine) —> Modules 5/6/7 driven by state
- Module 3 (Global Vars) —> Shared by all modules
- Module 4 (NVS Storage) —> Module 5 (load at boot), Module 8 (call on web save)
- Module 5 (WiFi Events) —> Module 6 (trigger reconnect), Module 7 (trigger fallback)
- Module 6 (WiFi Reconnect) —> Module 7 (enter fallback after failure)
- Module 7 (AP Fallback) —> Module 8 (start web server)
- Module 8 (Web Portal) —> Module 4 (save credentials to NVS)



Module 1: Configuration

Responsibility: Centralizes all user-adjustable parameters. Modify behavior by changing only here, without touching other modules.

```
1 //--- AP Fallback Hotspot (fixed, so users always know what to connect to) ---
2 const char* AP_SSID  = "ESP32_Setup"; // Fallback AP name
3 const char* AP_PASSWORD = "12345678"; // Fallback AP password (>= 8 chars)
4
5 //--- Pin Definitions ---
6 #define LED_BUILTIN 2
7
8 //--- Timing Parameters (all in milliseconds) ---
9 const unsigned long CONNECT_TIMEOUT  = 15000; // STA connect timeout
10 const unsigned long RECONNECT_MIN_MS = 5000; // Initial reconnect interval
11 const unsigned long RECONNECT_MAX_MS = 60000; // Max reconnect interval
12 const unsigned long STATUS_PRINT_MS  = 5000; // Status print interval
13 const unsigned long WDT_TIMEOUT_MS   = 300000; // Watchdog restart (5 min in FAIL)
14 const unsigned long AP_RETRY_STA_MS  = 120000; // Retry STA from AP every 2 min
15
16 //--- Max reconnect attempts before falling back to AP ---
17 const int MAX_RECONNECT_ATTEMPTS = 6;
```

Code Explanation:

Parameter	Meaning	Adjustment Suggestion
AP_SSID / AP_PASSWORD	Fallback hotspot info, hardcoded so users know which to connect	Change to recognizable name
LED_BUILTIN	LED pin number	May differ on different boards (GPIO0/4/33)
CONNECT_TIMEOUT	First STA connect timeout (15 sec)	Increase to 30 sec for weak signal
RECONNECT_MIN_MS	Min reconnect interval (5 sec)	Can reduce to 3 sec for faster response
RECONNECT_MAX_MS	Max reconnect interval (60 sec)	Can reduce to 30 sec for more aggressive
WDT_TIMEOUT_MS	Watchdog timeout (5 min)	Too short causes frequent reboots
AP_RETRY_STA_MS	STA retry period in AP mode (2 min)	Longer saves power, shorter recovers faster
MAX_RECONNECT_ATTEMPTS	How many failures before AP fallback	Larger is more persistent, smaller falls back faster



Design point: AP hotspot info is hardcoded, so no matter how STA fails, users know to connect to [ESP32_Setup](#) . STA credentials are stored in NVS and can be dynamically modified via webpage (see Module 4).

Module 2: State Machine Definition

Responsibility: Defines all possible states and the current state variable, serving as the "skeleton" of the entire program.

```
1 enum WiFiState {
2   STATE_BOOT,    // Just powered on
3   STATE_STA_TRYING, // Trying to connect in STA mode
4   STATE_STA_OK,   // STA connected successfully
5   STATE_STA_LOST, // STA connection lost, trying to recover
6   STATE_AP_FALLBACK, // STA failed, fell back to AP mode (web portal active)
7   STATE_FAIL     // Total failure (should not normally happen)
8 };
9
10 WiFiState state = STATE_BOOT;
11
```

```

12 // Convert state enum to readable string (for logging and web display)
13 String stateToString(WiFiState s){
14     switch(s){
15         case STATE_BOOT:    return "BOOT";
16         case STATE_STA_TRYING: return "STA_TRYING";
17         case STATE_STA_OK:   return "STA_OK";
18         case STATE_STA_LOST: return "STA_LOST";
19         case STATE_AP_FALLBACK: return "AP_FALLBACK";
20         case STATE_FAIL:    return "FAIL";
21         default:            return "UNKNOWN";
22     }
23 }

```

[Back to Elegoo](#)

Code Explanation:

- `enum WiFiState` : Defines all states using enum, far more readable than using numbers 0/1/2
- `state` : Global variable recording current state (declared in Module 3, defined here)
- `stateToString()` : Converts state to string for serial logging and web display



Beginner tip: Using names (like `STA_OK`) instead of numbers (like `2`) for states makes code immediately clear about "what it's doing now".

Module 3: Global Objects & Variables

Responsibility: Declares resources shared across all modules. These variables pass information between modules.

```

1 // --- Library Objects ---
2 WebServer server(80);
3 Preferences prefs;
4
5 // --- Stored Credentials (loaded from NVS at boot) ---
6 String staSSID = "";
7 String staPassword = "";
8
9 // --- Timing Variables ---
10 unsigned long lastReconnectAttempt = 0;
11 unsigned long reconnectInterval = RECONNECT_MIN_MS;
12 unsigned long lastStatusPrint = 0;
13 unsigned long stateEnterTime = 0;
14 unsigned long lastBlinkToggle = 0;
15
16 // --- Statistics (for diagnostics) ---
17 int reconnectAttempts = 0;
18 int successfulConnects = 0;
19 int disconnectCount = 0;
20 bool ledState = false;

```

Code Explanation:

Variable	Type	Purpose	Using Modules
<code>server</code>	WebServer	HTTP server object, port 80	Modules 7, 8
<code>prefs</code>	Preferences	NVS storage object	Module 4
<code>staSSID / staPassword</code>	String	STA credentials loaded from NVS	Modules 4, 5, 8
<code>lastReconnectAttempt</code>	unsigned long	Last reconnect timestamp	Module 6
<code>reconnectInterval</code>	unsigned long	Current reconnect interval (grows exponentially)	Module 6
<code>stateEnterTime</code>	unsigned long	Time when entered current state	Modules 6, 7
<code>reconnectAttempts</code>	int	Retry count since last disconnect	Modules 5, 6
<code>successfulConnects</code>	int	Cumulative successful connection count	Modules 5, 9

Variable	Type	Purpose	Using Modules
<code>disconnectCount</code>	int	Cumulative disconnect count	Modules 5, 9



Key change: Unlike the previous version, `staSSID` / `staPassword` are no longer hardcoded constants but `String` type variables loaded from NVS, modifiable via webpage.

Module 4: NVS Storage

Responsibility: Handles persistent storage of WiFi credentials. Data survives power off, auto-read on reboot.

```

1  void loadCredentials() {
2    prefs.begin("wifi_creds", true); // read-only
3    staSSID = prefs.getString("ssid", "");
4    staPassword = prefs.getString("pass", "");
5    prefs.end();
6
7    Serial.print("[NVS] Loaded SSID: ");
8    Serial.println(staSSID.length() > 0 ? staSSID : "(empty)");
9  }
10
11 void saveCredentials(String ssid, String pass) {
12   prefs.begin("wifi_creds", false); // read-write
13   prefs.putString("ssid", ssid);
14   // Only update password if user provided a non-empty value
15   // (so "leave blank to keep current" works on the web form)
16   if (pass.length() > 0) {
17     prefs.putString("pass", pass);
18   }
19   prefs.end();
20
21   Serial.println("[NVS] Credentials saved");
22   Serial.print("  SSID: ");
23   Serial.println(ssid);
24 }
```

Code Explanation:

`loadCredentials()` — Called once at boot:

- `prefs.begin("wifi_creds", true)` : Opens namespace `wifi_creds` , second parameter `true` means read-only
- `getString("ssid", "")` : Reads key `ssid` , returns default value `""` if not exists
- `prefs.end()` : Closes namespace, **must be called**

`saveCredentials()` — Called when webpage saves:

- `prefs.begin("wifi_creds", false)` : Second parameter `false` means read-write mode
- `putString()` : Writes key-value pair
- **"Leave blank to keep" logic for password:** Only overwrites if user entered non-empty password, so users only changing SSID don't need to re-enter password

Call timing:

- `loadCredentials()` : Called in `setup()` of Module 10
- `saveCredentials()` : Called in `handleSave()` of Module 8



NVS simple analogy: Like ESP32's "little notebook", `loadCredentials()` is "open notebook to read records", `saveCredentials()` is "write new records in notebook".

Module 5: WiFi Connection & Events

Responsibility: Handles WiFi event callbacks, is the "brain" of the entire program, senses WiFi status changes in real-time and drives state transitions.

5.1 WiFi Event Callback

```

1 void onWiFiEvent(WiFiEvent_t event) {
2   switch (event) {
3     case WiFiEvent_t::ARDUINO_EVENT_WIFI_STA_START:
4       Serial.println("[EVENT] STA started");
5       break;
6
7     case WiFiEvent_t::ARDUINO_EVENT_WIFI_STA_CONNECTED:
8       Serial.println("[EVENT] STA associated with AP");
9       break;
10
11    case WiFiEvent_t::ARDUINO_EVENT_WIFI_STA_GOT_IP:
12      // This is the real "connected" event - we have an IP address now
13      Serial.print("[EVENT] Got IP: ");
14      Serial.println(WiFi.localIP());
15      successfulConnects++;
16      reconnectInterval = RECONNECT_MIN_MS; // Reset backoff after success
17      switchState(STATE_STA_OK);
18      break;
19
20    case WiFiEvent_t::ARDUINO_EVENT_WIFI_STA_DISCONNECTED:
21      Serial.println("[EVENT] STA disconnected");
22      disconnectCount++;
23      if (state == STATE_STA_OK) {
24        switchState(STATE_STA_LOST);
25      }
26      break;
27
28    case WiFiEvent_t::ARDUINO_EVENT_WIFI_AP_START:
29      Serial.println("[EVENT] AP fallback started");
30      break;
31
32    default:
33      break;
34  }
35 }

```

Event Explanation:

Event	This Module's Handling
STA_START	Only logs
STA_CONNECTED	Only logs (no IP yet, can't count as success)
STA_GOT_IP	Key event — resets backoff counter, switches to STA_OK
STA_DISCONNECTED	Increments disconnect count, switches from STA_OK to STA_LOST
AP_START	Only logs



Common pitfall: `STA_CONNECTED` does not mean "connected"! It may still be in DHCP process, starting communication without IP will fail. Must wait for `STA_GOT_IP` to be truly usable.

5.2 State Transition Function

```

1 void switchState(WiFiState newState) {
2   Serial.print("[STATE] ");
3   Serial.print(stateToString(state));
4   Serial.print(" -> ");
5   Serial.println(stateToString(newState));
6
7   // Exit actions (cleanup when leaving a state)
8   if (state == STATE_AP_FALLBACK && newState != STATE_AP_FALLBACK) {
9     stopAPFallback(); // Stop web server and AP when leaving AP_FALLBACK
10  }
11
12  state = newState;
13  stateEnterTime = millis();
14
15 }

```

```

15 // Entry actions (setup when entering a state)
16 if (newState == STATE_STA_TRYING) {
17     Serial.println("[ACTION] Connecting to STA: " + staSSID);
18     WiFi.mode(WIFI_STA);
19     WiFi.begin(staSSID.c_str(), staPassword.c_str());
20     lastReconnectAttempt = millis();
21     reconnectAttempts++;
22 } else if (newState == STATE_AP_FALLBACK) {
23     startAPFallback();
24 }
25 }

```

[Back to Elegoo](#)

Code Explanation:

This is the **single entry point function** for the state machine; all state transitions go through here, making it easy to add logging and guard conditions.

- ▶ **Exit Action:** When leaving `AP_FALLBACK`, calls `stopAPFallback()` (Module 7) to clean up web server and AP
- ▶ **Entry Action:**
 - ▶ Entering `STA_TRYING`: Initiates new WiFi connection
 - ▶ Entering `AP_FALLBACK`: Calls `startAPFallback()` (Module 7)



State machine best practice: All state transitions go through this one function, avoiding scattered direct assignments. This is "symmetric design": entry actions have corresponding exit actions.

Module 6: WiFi Reconnection

Responsibility: Handles reconnection logic after STA connection failure or disconnect, uses exponential backoff to avoid overwhelming the router.

6.1 STATE_STA_TRYING Handler

```

1 void handleSTATrying() {
2     unsigned long now = millis();
3     if (now - lastReconnectAttempt >= CONNECT_TIMEOUT) {
4         Serial.println("[ACTION] STA connect timeout, entering AP fallback");
5         switchState(STATE_AP_FALLBACK);
6     }
7     // Successful connection is handled by the GOT_IP event in Module 5
8 }

```

Logic: If no `GOT_IP` event received within 15 seconds (possibly wrong password, router not responding), gives up and switches to AP fallback. Success case is handled by Module 5's event callback; this only handles timeout.

6.2 STATE_STA_LOST Handler

```

1 void handleSTALost() {
2     unsigned long now = millis();
3     if (now - lastReconnectAttempt >= reconnectInterval) {
4         lastReconnectAttempt = now;
5         reconnectAttempts++;
6         Serial.print("[ACTION] Reconnect attempt #");
7         Serial.print(reconnectAttempts);
8         Serial.print(" (interval ");
9         Serial.print(reconnectInterval / 1000);
10        Serial.println("s)");
11
12        WiFi.disconnect();
13        WiFi.begin(staSSID.c_str(), staPassword.c_str());
14
15        // Exponential backoff: double the interval up to the max
16        reconnectInterval = min(reconnectInterval * 2, RECONNECT_MAX_MS);
17
18        // If too many failed attempts, fall back to AP portal
19        if (reconnectAttempts >= MAX_RECONNECT_ATTEMPTS) {
20            Serial.println("[ACTION] Too many failed attempts, entering AP fallback");
21            switchState(STATE_AP_FALLBACK);
22        }
23    }
24 }

```

Core Logic:

[Back to Elegoo](#)

1. **Non-blocking timing:** Uses `millis()` to determine if it's time to retry, no `delay()`
2. **Disconnect before reconnect:** `WiFi.disconnect()` clears old state, avoids underlying state confusion
3. **Exponential Backoff:** After each failure, retry interval doubles

Exponential backoff illustration:

```
1 | 1st failure → wait 5 sec to retry
2 | 2nd failure → wait 10 sec to retry
3 | 3rd failure → wait 20 sec to retry
4 | 4th failure → wait 40 sec to retry
5 | 5th failure → wait 60 sec to retry (reached max)
6 | 6th failure → switch to AP fallback
```

Why use exponential backoff?

- Avoid overwhelming router with frequent reconnects (router may be rebooting)
- Reduce wireless channel usage, yield to other devices
- A classic algorithm in the TCP/IP protocol family



Module dependency: After 6 failed reconnection attempts, calls Module 5's `switchState(STATE_AP_FALLBACK)` to enter fallback mode.

Module 7: AP Fallback

Responsibility: After STA fails continuously, switches to AP mode and starts web portal, letting users modify configuration via browser.

7.1 Start AP Fallback

```
1 | void startAPFallback() {
2 |   Serial.println("[ACTION] Starting AP fallback: " + String(AP_SSID));
3 |   WiFi.mode(WIFI_AP);
4 |   bool ok = WiFi.softAP(AP_SSID, AP_PASSWORD, 6, 0, 4);
5 |   if (ok) {
6 |     Serial.print("  AP IP: ");
7 |     Serial.println(WiFi.softAPIP());
8 |     Serial.println("  Open http://192.168.4.1 in a browser to configure WiFi");
9 |     setupWebServer(); // Module 8: register web routes
10 |    server.begin(); // Start HTTP server
11 |    Serial.println("  Web server started on port 80");
12 |   } else {
13 |     Serial.println("  AP start failed, entering FAIL state");
14 |     switchState(STATE_FAIL);
15 |   }
16 | }
```

Logic:

1. `WiFi.mode(WIFI_AP)` : Switch to pure AP mode
2. `WiFi.softAP(...)` : Start hotspot (SSID/password from Module 1)
3. `setupWebServer()` : Call Module 8 to register web routes
4. `server.begin()` : Start HTTP server
5. If AP start fails, enter `FAIL` state (see 7.4)

7.2 Stop AP Fallback

```
1 | void stopAPFallback() {
2 |   server.stop();
3 |   server.close();
4 |   WiFi.softAPdisconnect(true);
5 |   Serial.println("[ACTION] AP fallback stopped");
6 | }
```

Logic: Called when leaving `AP_FALLBACK` state (triggered by Module 5's `switchState`), cleans up web server and disconnects AP, avoiding WiFi mode confusion.

```

1 | void handleAPFallback() {
2 |   // Serve web portal clients (must be called frequently)
3 |   server.handleClient();
4 |
5 |   unsigned long now = millis();
6 |
7 |   // Auto-retry STA only if NO user is currently connected to the AP
8 |   // (so we don't kick off someone who is configuring via the web page)
9 |   if (now - stateEnterTime >= AP_RETRY_STA_MS) {
10 |    int clients = WiFi.softAPgetStationNum();
11 |    if (clients == 0) {
12 |      Serial.println("[ACTION] Auto-retrying STA from AP fallback (no clients connected)");
13 |      reconnectAttempts = 0;
14 |      reconnectInterval = RECONNECT_MIN_MS;
15 |      switchState(STATE_STA_TRYING);
16 |    } else {
17 |      Serial.print("[ACTION] Web client connected (");
18 |      Serial.print(clients);
19 |      Serial.println("), deferring STA retry");
20 |      stateEnterTime = now; // Reset timer to check again later
21 |    }
22 |  }
23 | }

```

Key Logic:

1. `server.handleClient()` : Must be called in each loop iteration to handle browser requests (Module 8)
2. **Smart retry**: Checks every 2 minutes whether to retry STA
 - **No client connected**: Auto-switch back to STA mode to retry (router may have recovered)
 - **Client connected: Defer retry**, avoiding kicking off users configuring via webpage

Why this matters? If a user is entering password on the webpage and ESP32 suddenly switches to STA mode, the AP hotspot disappears, the user's phone disconnects, and form submission fails. By checking `softAPgetStationNum()`, ensures mode is never switched while someone is online.

7.4 STATE_FAIL Handler (Watchdog)

```

1 | void handleFail() {
2 |   unsigned long now = millis();
3 |   if (now - stateEnterTime >= WDT_TIMEOUT_MS) {
4 |     Serial.println("[ACTION] Watchdog timeout, restarting ESP32...");
5 |     delay(100);
6 |     ESP.restart();
7 |   }
8 | }

```

Logic: If even AP mode can't start (very rare), waits 5 minutes then hard reboots ESP32. This is the **last line of defense**.

Module 8: Web Configuration Portal

Responsibility: HTTP server that lets users configure WiFi credentials via browser. This is the "user interaction interface".

8.1 Route Registration

```

1 | void setupWebServer() {
2 |   server.on("/", HTTP_GET, handleRoot);
3 |   server.on("/save", HTTP_POST, handleSave);
4 |   server.on("/reboot", HTTP_POST, handleReboot);
5 |   server.on("/status", HTTP_GET, handleStatus);
6 |   server.onNotFound([]() {
7 |     server.send(404, "text/plain", "404: Not Found");
8 |   });
9 | }

```

Route Table:

Path	Method	Function
/	GET	Display config homepage (HTML form)
/save	POST	Save new WiFi credentials and reboot
/reboot	POST	Reboot only (no credential change)
/status	GET	Return JSON format status info (for debugging)
Other	-	Return 404

Simple analogy: Like different windows at a restaurant — `/` is the "order window", `/save` is the "place order window", `/reboot` is the "clear table window".

8.2 Homepage Handler `GET /`

```

1 | void handleRoot() {
2 |   String html = getHTMLPage();
3 |   server.send(200, "text/html", html);
4 | }
```

Returns HTML configuration page (see 8.6).

8.3 Save Credentials `POST /save`

```

1 | void handleSave() {
2 |   String newSSID = server.arg("ssid");
3 |   String newPass = server.arg("pass");
4 |
5 |   newSSID.trim();
6 |
7 |   if (newSSID.length() == 0) {
8 |     server.send(400, "text/html", "<h2>Error: SSID cannot be empty</h2><a href='/'>Back</a>");
9 |     return;
10 |  }
11 |
12 |   // Save to NVS (Module 4)
13 |   saveCredentials(newSSID, newPass);
14 |
15 |   // Update in-memory copies
16 |   staSSID = newSSID;
17 |   if (newPass.length() > 0) {
18 |     staPassword = newPass;
19 |   }
20 |
21 |   // Respond then reboot
22 |   String html = "<!DOCTYPE html><html><head>...";
23 |   html += "<h2>Saved!</h2><p>SSID: <b>" + newSSID + "</b></p>";
24 |   html += "<p>ESP32 is rebooting... Please reconnect to your WiFi after ~10 seconds.</p>";
25 |   html += "</body></html>";
26 |   server.send(200, "text/html", html);
27 |
28 |   Serial.println("[WEB] Credentials saved via web portal, rebooting...");
29 |   delay(500);
30 |   ESP.restart();
31 | }
```

Flow:

1. `server.arg("ssid")` : Get user-entered SSID from form
2. `newSSID.trim()` : Remove leading/trailing spaces
3. **Validation:** SSID cannot be empty
4. `saveCredentials()` : Save to NVS (calls Module 4)
5. Update in-memory variables
6. Send "Saved, rebooting" response page
7. `delay(500)` : Wait for response to be sent



Key: Must `server.send()` the response first, then `ESP.restart()` . If rebooting directly, user's browser sees "connection interrupted" and doesn't know if save succeeded.

8.4 One-Click Reboot `POST /reboot`

```

1  void handleReboot(){
2      String html = "<!DOCTYPE html><html>...";
3      html += "<h2>Rebooting...</h2><p>ESP32 is restarting. Please wait ~10 seconds.</p>";
4      html += "</body></html>";
5      server.send(200, "text/html", html);
6
7      Serial.println("[WEB] Reboot requested via web portal");
8      delay(500);
9      ESP.restart();
10 }

```

Function: Doesn't modify any credentials, only reboots ESP32. Used to manually let ESP32 retry connection after router reboot, or to test reboot functionality.

8.5 Status Query `GET /status` (JSON)

```

1  void handleStatus(){
2      String json = "{";
3      json += "\"state\": \"" + stateToString(state) + "\", ";
4      json += "\"uptime_s\": " + String(millis() / 1000) + ", ";
5      json += "\"reconnect_attempts\": " + String(reconnectAttempts) + ", ";
6      // ... more fields
7      json += "}";
8      server.send(200, "application/json", json);
9  }

```

Returns JSON format status info, convenient for advanced users or third-party programs to read. Browser accessing <http://192.168.4.1/status> will see:

```

1  {"state":"AP_FALLBACK","uptime_s":45,"reconnect_attempts":6,...}

```

8.6 HTML Page Generator

```

1  String getHTMLPage(){
2      String html = "<!DOCTYPE html><html lang='zh-CN'>";
3      html += "<head><meta charset='UTF-8'>";
4      html += "<meta name='viewport' content='width=device-width, initial-scale=1'>";
5      html += "<title>ESP32 WiFi Configuration</title>";
6      html += "<style> /* ... mobile-friendly CSS ... */ </style>";
7      html += "</head><body>";
8      html += "<div class='card'>";
9      html += "<h2>ESP32 WiFi Settings</h2>";
10
11     html += "<form action='/save' method='POST'>";
12     html += "<label>WiFi Name (SSID)</label>";
13     html += "<input type='text' name='ssid' value='\" + staSSID + \"' required>";
14     html += "<label>WiFi Password</label>";
15     html += "<input type='text' name='pass' placeholder='Leave blank to keep current'>";
16     html += "<button type='submit' class='btn-save'>Save and Reboot</button>";
17     html += "</form>";
18
19     html += "<form action='/reboot' method='POST'>";
20     html += "<button type='submit' class='btn-reboot'>Reboot Only (No Change)</button>";
21     html += "</form>";
22
23     html += "<div class='stats'>Uptime: " + String(millis() / 1000) + "s | ...</div>";
24     html += "</div></body></html>";
25     return html;
26 }

```

```

1 | _____
2 | | ESP32 WiFi Settings | |
3 | |_____
4 | | WiFi Name (SSID) | |
5 | |_____
6 | | | MyHomeWiFi | | | ← Current SSID (pre-filled)
7 | |_____
8 | | WiFi Password | |
9 | |_____
10 | | | Leave blank to keep current | | | ← Blank means no change
11 | |_____
12 | | [ Save and Reboot (green) ] | | ← POST /save
13 | | [ Reboot Only (red) ] | | ← POST /reboot
14 | | Uptime: 45s | Reconnects: 6 | | ← Runtime stats
15 | |_____

```

Code Explanation:

- ▶ `<meta name='viewport' ...>` : Makes webpage auto-adapt width on mobile (**responsive design**)
- ▶ `value="" + staSSID + ""` : Pre-fills input box with currently saved SSID
- ▶ `placeholder='Leave blank to keep current'` : Placeholder hint
- ▶ Two independent `<form>` : One submits to `/save` , one to `/reboot`
- ▶ Bottom `stats` : Shows runtime statistics



Module dependency: `handleSave()` calls Module 4's `saveCredentials()` to save credentials, then `ESP.restart()` to reboot.

Module 9: Diagnostics & LED

Responsibility: Serial status reporting and LED blink patterns, letting users understand device status without a computer.

9.1 Serial Status Print

```

1 | void printStatus(){
2 |   Serial.println("\n----- WiFi Status -----");
3 |   Serial.print("State   :");
4 |   Serial.println(stateToString(state));
5 |
6 |   if (state == STATE_STA_OK || state == STATE_STA_LOST || state == STATE_STA_TRYING){
7 |     Serial.print("Mode     : STA");
8 |     Serial.print(" | SSID: ");
9 |     Serial.println(WiFi.SSID());
10 |    Serial.print("IP       :");
11 |    Serial.println(WiFi.localIP());
12 |    Serial.print("RSSI (dBm)  :");
13 |    Serial.println(WiFi.RSSI());
14 |   } else if (state == STATE_AP_FALLBACK){
15 |     Serial.print("Mode     : AP");
16 |     Serial.print(" | SSID: ");
17 |     Serial.println(AP_SSID);
18 |     Serial.print("AP IP    :");
19 |     Serial.println(WiFi.softAPIP());
20 |     Serial.print("Connected clients :");
21 |     Serial.println(WiFi.softAPgetStationNum());
22 |     Serial.println("Web portal   : http://192.168.4.1");
23 |   }
24 |
25 |   Serial.print("Reconnect attempts:");
26 |   Serial.print(reconnectAttempts);
27 |   Serial.print(" | Successful connects:");
28 |   Serial.print(successfulConnects);
29 |   Serial.print(" | Disconnects:");
30 |   Serial.println(disconnectCount);
31 |   Serial.print("Saved SSID   :");
32 |   Serial.println(staSSID);
33 |   Serial.print("Uptime (s)   :");
34 |   Serial.println(millis() / 1000);
35 |   Serial.println("-----\n");
36 | }

```

Diagnostic value: Periodically reviewing this data after deployment lets you judge if the device is healthy: [Back to Elegoo](#)

- ▶ [Disconnects](#) increasing dozens of times per hour → poor signal, may need antenna or repositioning
- ▶ [Successful connects](#) continuously increasing → frequent disconnects and reconnects, unstable environment
- ▶ [RSSI < -80](#) → weak signal, may need repositioning

9.2 LED Status Indicator

```
1 void updateLED() {
2   unsigned long now = millis();
3   unsigned long blinkInterval = 0;
4
5   switch (state) {
6     case STATE_STA_OK:
7       digitalWrite(LED_BUILTIN, HIGH); // Solid on
8       return;
9
10    case STATE_STA_TRYING:
11      blinkInterval = 500; // Slow blink
12      break;
13
14    case STATE_STA_LOST:
15      blinkInterval = 150; // Fast blink
16      break;
17
18    case STATE_AP_FALLBACK: {
19      // Double-blink: on 100ms, off 100ms, on 100ms, off 500ms
20      unsigned long phase = (now - lastBlinkToggle) % 800;
21      bool on = (phase < 100) || (phase >= 200 && phase < 300);
22      digitalWrite(LED_BUILTIN, on ? HIGH : LOW);
23      return;
24    }
25
26    case STATE_FAIL:
27    case STATE_BOOT:
28    default:
29      digitalWrite(LED_BUILTIN, LOW); // Off
30      return;
31    }
32
33    if (blinkInterval > 0 && now - lastBlinkToggle >= blinkInterval) {
34      lastBlinkToggle = now;
35      ledState = !ledState;
36      digitalWrite(LED_BUILTIN, ledState ? HIGH : LOW);
37    }
38  }
```

LED Pattern Table:

State	LED Behavior	Meaning
BOOT	Off	Just started
STA_TRYING	Slow blink (500ms period)	Connecting to router
STA_OK	Solid on	Connection normal
STA_LOST	Fast blink (150ms period)	Disconnected, reconnecting
AP_FALLBACK	Double-blink (on-off-on-long off)	Fell back to AP mode, needs configuration
FAIL	Off	Total failure, waiting for watchdog



Practical value: After deployment without computer, you can judge status just by looking at LED. Double-blink pattern is particularly recognizable, suitable for indicating "user intervention needed".

Module 10: Main Program

Responsibility: `setup()` and `loop()` , assembling all modules together.

[Back to Elegoo](#)

10.1 `setup()` Initialization

```
1 void setup(){
2   Serial.begin(115200);
3   delay(1000);
4
5   Serial.println("\n=====");
6   Serial.println(" ESP32 WiFi Stability + Web Portal");
7   Serial.println("=====");
8
9   pinMode(LED_BUILTIN, OUTPUT);
10  digitalWrite(LED_BUILTIN, LOW);
11
12  // Module 4: Load saved credentials from NVS
13  loadCredentials();
14
15  // Module 5: Register WiFi event callback + disable auto-reconnect
16  WiFi.onEvent(onWiFiEvent);
17  WiFi.setAutoReconnect(false); // We manage reconnection manually
18
19  // Decide initial state based on whether credentials exist
20  if (staSSID.length() == 0) {
21    // First-time use: no credentials saved -> go straight to AP portal
22    Serial.println("[BOOT] No saved credentials, starting AP config portal");
23    switchState(STATE_AP_FALLBACK);
24  } else {
25    // Credentials exist -> try STA first
26    switchState(STATE_STA_TRYING);
27  }
28 }
```

Step Analysis:

1. **Serial and LED initialization**
2. **Module 4:** `loadCredentials()` loads credentials from NVS
3. **Module 5:** Register event callback + disable auto-reconnect
4. **Key decision:** Check if credentials exist in NVS
 - **None (first use):** Directly enter `AP_FALLBACK` , start config portal
 - **Exist:** Enter `STA_TRYING` , try connecting with saved credentials



First-use experience: User gets a newly flashed ESP32, powers it on, LED double-blinks (indicating AP mode), phone finds `ESP32_Setup` hotspot, connects, opens `192.168.4.1` in browser, enters home WiFi info, clicks save and reboot, done.

10.2 `loop()` Main Loop

```
1 void loop(){
2   unsigned long now = millis();
3
4   // Drive the state machine (dispatches to Modules 6, 7, 8)
5   switch (state){
6     case STATE_STA_TRYING: handleSTATrying(); break; // Module 6
7     case STATE_STA_OK:      break; // Idle; events drive transitions
8     case STATE_STA_LOST:   handleSTALost(); break; // Module 6
9     case STATE_AP_FALLBACK: handleAPFallback(); break; // Module 7 + 8
10    case STATE_FAIL:      handleFail(); break; // Module 7
11    case STATE_BOOT:      break;
12  }
13
14  // Module 9: Periodic status print
15  if (now - lastStatusPrint >= STATUS_PRINT_MS){
16    lastStatusPrint = now;
17    printStatus();
18  }
19
20  // Module 9: LED indicator update
21  updateLED();
22
23 }
```

```

24 | delay(10);
    | }

```

[Back to Elegoo](#)

Code Explanation:

- Main loop is very concise: calls the corresponding module's handler based on current state
- `STATE_STA_OK` does nothing — because already connected, just waits for Module 5's event callback to notify disconnect
- Periodically calls Module 9 to print status and update LED
- `delay(10)` gives CPU breathing room, but doesn't affect event response (events are triggered asynchronously by underlying layer)



Design essence: This "event-driven + state machine" structure clearly separates "when to do what" (state machine) from "how to do it" (each module). Each module minds its own business, communicates via state variables. Code readability and maintainability are far superior to cramming all logic into `loop()`.

Operation Steps Guide

Step 1: First Flash (No Code Modification Needed)

Open the ino file in the resource package and do not make any changes for the time being.



Important: Unlike the previous version, this version **does not require entering WiFi credentials in the code**. On first boot, ESP32 will automatically enter the config portal; you fill in via webpage.

Step 2: Observe First Boot

Open the Serial Monitor (baud rate 115200), first boot should show:

```

1 | =====
2 | ESP32 WiFi Stability + Web Portal
3 | =====
4 | [NVS] Loaded SSID: (empty)
5 | [BOOT] No saved credentials, starting AP config portal
6 | [STATE] BOOT -> AP_FALLBACK
7 | [ACTION] Starting AP fallback: ESP32_Setup
8 |   AP IP: 192.168.4.1
9 |   Open http://192.168.4.1 in a browser to configure WiFi
10 |   Web server started on port 80
11 | [EVENT] AP fallback started

```

At this point, LED should **double-blink**.

Step 3: Connect to ESP32 Hotspot

1. Open WiFi list on phone/computer
2. Find WiFi named `ESP32_Setup`
3. Enter password `12345678` to connect
4. Phone shows "Connected but no internet access" — normal, continue to next step

Step 4: Open Configuration Webpage

1. Open browser on phone/computer (Chrome, Safari, etc.)
2. Enter in address bar: `http://192.168.4.1`
3. Press Enter, should see configuration page:

```

1 | _____
2 | | ESP32 WiFi Settings | _____
3 | |_____
4 | | WiFi Name (SSID) | _____
5 | | _____ | _____
6 | | | | _____ | _____
7 | | _____ | _____
8 | | WiFi Password | _____
9 | | _____ | _____
10 | | Leave blank to keep current | | _____
11 | | _____ | _____
12 | | [ Save and Reboot (green) ] | _____
13 |

```

Step 5: Enter WiFi Info and Save

1. Enter your home WiFi name in "WiFi Name (SSID)" input box
2. Enter WiFi password in "WiFi Password" input box
3. Click green button **"Save and Reboot"**
4. Webpage shows "Saved! ESP32 is rebooting..."
5. Wait about 10 seconds for ESP32 to reboot

Step 6: Verify Successful Connection

After ESP32 reboots, it will read the just-saved credentials from NVS and try to connect to your home WiFi. Serial Monitor should show:

```

1  [NVS] Loaded SSID: MyHomeWiFi
2  [STATE] BOOT -> STA_TRYING
3  [ACTION] Connecting to STA: MyHomeWiFi
4  [EVENT] STA started
5  [EVENT] STA associated with AP
6  [EVENT] Got IP: 192.168.1.105
7  [STATE] STA_TRYING -> STA_OK

```

At this point, LED should be **solid on**, indicating successful connection.

Step 7: Test Password Change Scenario (Simulating Real Use)

1. Suppose your home router password was changed, ESP32 can't connect
2. Observe ESP32 serial: after 6 failed reconnection attempts, automatically enters [AP_FALLBACK](#)
3. LED double-blinks, reminding you to reconfigure
4. Phone connects to [ESP32_Setup](#) hotspot
5. Open [192.168.4.1](#) in browser
6. Enter new WiFi password, click save and reboot
7. ESP32 connects successfully with new password

No computer, no Arduino IDE, no coding needed throughout.

Step 8: Test One-Click Reboot

1. Connect to [ESP32_Setup](#) hotspot (ESP32 must be in AP mode)
2. Open [192.168.4.1](#)
3. Click red button **"Reboot Only"**
4. ESP32 immediately reboots, no configuration changes

Expected Serial Output

Scenario 1: First Boot (No Saved Credentials)

```

1  =====
2  ESP32 WiFi Stability + Web Portal
3  =====
4  [NVS] Loaded SSID: (empty)
5  [BOOT] No saved credentials, starting AP config portal
6  [STATE] BOOT -> AP_FALLBACK
7  [ACTION] Starting AP fallback: ESP32_Setup
8  AP IP: 192.168.4.1
9  Open http://192.168.4.1 in a browser to configure WiFi
10 Web server started on port 80
11 [EVENT] AP fallback started
12
13 ----- WiFi Status -----
14 State      : AP_FALLBACK
15 Mode       : AP | SSID: ESP32_Setup
16 AP IP      : 192.168.4.1
17 Connected clients : 0
18 Web portal  : http://192.168.4.1
19 Reconnect attempts: 0 | Successful connects: 0 | Disconnects: 0
20 Saved SSID  :
21

```

Scenario 2: After Saving Credentials via Webpage and Reboot

```

1 | =====
2 | ESP32 WiFi Stability + Web Portal
3 | =====
4 | [NVS] Loaded SSID: MyHomeWiFi
5 | [STATE] BOOT -> STA_TRYING
6 | [ACTION] Connecting to STA: MyHomeWiFi
7 | [EVENT] STA started
8 | [EVENT] STA associated with AP
9 | [EVENT] Got IP: 192.168.1.105
10 | [STATE] STA_TRYING -> STA_OK

```

Scenario 3: User Modifies Password via Webpage

(Serial side)

```

1 | [WEB] Credentials saved via web portal, rebooting...

```

(After reboot)

```

1 | [NVS] Loaded SSID: MyHomeWiFi
2 | [STATE] BOOT -> STA_TRYING
3 | [ACTION] Connecting to STA: MyHomeWiFi
4 | [EVENT] Got IP: 192.168.1.105
5 | [STATE] STA_TRYING -> STA_OK

```

Scenario 4: One-Click Reboot

```

1 | [WEB] Reboot requested via web portal
2 | (After reboot, starts from setup again)

```

FAQ

Q1: Why doesn't first boot require entering WiFi info in the code?

A: Because this version uses NVS persistent storage. On first boot, it detects NVS is empty and directly enters AP config portal. User fills in via webpage, saves to NVS. Every subsequent boot reads from NVS. This is the standard approach for industrial-grade products, far more flexible than "hardcoding in code".

Q2: Can't open webpage 192.168.4.1 ?

A: Please check:

- ▶ Is phone actually connected to [ESP32_Setup](#) hotspot? (Sometimes phone auto-switches back to 4G)
- ▶ Is "Mobile Data" enabled on phone? Some phones, when connected to a hotspot without internet, browser uses 4G instead of WiFi. **Please temporarily disable mobile data**
- ▶ Is browser address correct? Must be <http://192.168.4.1> (note the <http://>)
- ▶ Some phone browsers auto-redirect to <https://> , please manually change back to <http://>

Q3: After saving and rebooting, still can't connect to WiFi?

A: Please check:

- ▶ Is the SSID entered on webpage correct? (Watch for case, spaces)
- ▶ Is the password correct?
- ▶ Is the router 2.4GHz? (ESP32 doesn't support 5GHz)
- ▶ Check [\[EVENT\]](#) logs via serial to determine if it's wrong password or weak signal

Q4: What happens if password field is left blank?

A: Blank means "keep original password, only change SSID". This is the design in [saveCredentials\(\)](#) :

```

1 | if (pass.length() > 0) {
2 |   prefs.putString("pass", pass);

```

Only overwrites if user entered a new password. This way, users only changing SSID don't need to re-enter password.

Q5: Will auto-retry STA in AP mode kick me off?

A: No. The code checks `WiFi.softAPgetStationNum()`, as long as there's a client connected, it won't switch modes. Only retries when no one is connected.

Q6: Can the webpage have password protection? To prevent others from tampering?

A: Yes, but omitted in this tutorial for simplicity. You can add HTTP Basic Auth in `setupWebServer()`:

```
1 | if (!server.authenticate("admin", "mypass")){
2 |   return server.requestAuthentication();
3 | }
```

Note that AP password and webpage password are two different layers of protection, recommended to set both.

Q7: Can data in NVS be cleared?

A: Yes, add in code:

```
1 | prefs.begin("wifi_creds", false);
2 | prefs.clear();
3 | prefs.end();
```

Or use `esptool.py erase_region` command to clear NVS partition. The simplest method is to re-flash firmware (doesn't clear NVS, needs separate erase).

Troubleshooting Table

Problem	Possible Cause	Solution
First boot goes directly to STA_TRYING	Previous code left NVS data	Use <code>prefs.clear()</code> to clear NVS
Webpage won't open	Phone using 4G not WiFi	Disable mobile data
Webpage won't open	Address uses https	Change to http://
Can't connect after save	Wrong SSID/password	Check case, reconfigure
Can't connect after save	Router is 5GHz	Use 2.4GHz
AP mode LED doesn't double-blink	Wrong pin definition	Check <code>LED_BUILTIN</code>
Webpage button no response	Browser cache	Force refresh (Ctrl+F5)
<code>ESP.restart()</code> doesn't work	Hardware fault	Check power supply

Modification Suggestions

1. Modify AP Hotspot Name and Password

```
1 | const char* AP_SSID = "MyIoT_Setup"; // Change to recognizable name
2 | const char* AP_PASSWORD = "setup12345"; // Change to your password
```

2. Add Password Protection to Webpage (HTTP Basic Auth)

```
1 | void setupWebServer(){
2 |   server.on("/", HTTP_GET, [](){
3 |     if (!server.authenticate("admin", "esp32pass")){
4 |       return server.requestAuthentication();
5 |     }
6 |     handleRoot();
7 |   });
8 | }
9 |
```

```
// ... add protection to other routes too
}
```

[Back to Elegoo](#)

3. Scan Nearby WiFi List

Display nearby WiFi on homepage, users can select instead of typing:

```
1 String getWiFiScanOptions() {
2   String html = "<select name='ssid'>";
3   int n = WiFi.scanNetworks();
4   for (int i = 0; i < n; i++) {
5     html += "<option value='" + WiFi.SSID(i) + "'>";
6     html += WiFi.SSID(i) + " (" + WiFi.RSSI(i) + " dBm)";
7     html += "</option>";
8   }
9   html += "</select>";
10  return html;
11 }
```

4. Add Captive Portal (Force Redirect)

Make browser auto-popup config page after connecting to hotspot (no need to manually enter IP):

```
1 #include <DNSServer.h>
2 DNSServer dnsServer;
3 // In setup: dnsServer.start(53, "*", WiFi.softAPIP());
4 // In loop: dnsServer.processNextRequest();
```

5. Save Multiple WiFi Alternatives

Support storing 3 WiFi networks, try in priority order:

```
1 prefs.begin("wifi_creds", false);
2 prefs.putString("ssid1", "HomeWiFi");
3 prefs.putString("ssid2", "OfficeWiFi");
4 prefs.putString("ssid3", "PhoneHotspot");
5 prefs.end();
```

6. Add More Buttons to Webpage

- "Clear Config": Clear NVS, restore factory state
- "View Logs": Show recent disconnect records
- "Signal Scan": Show nearby WiFi list

Notes

Note 1: NVS Lifespan

NVS is based on flash, with limited erase/write cycles (about 100,000 times). Each credential save writes data, don't call `saveCredentials()` frequently in `loop()`. Normal use (occasional password changes) is more than sufficient.

Note 2: Webpage Security

This tutorial's web portal **has no password protection**, anyone who can connect to [ESP32_Setup](#) hotspot can modify configuration. In actual products you should:

- Set a strong AP password
- Add HTTP Basic Auth or token to webpage
- Close AP after configuration (this tutorial auto-switches back to STA)

Note 3: HTTP vs HTTPS

This tutorial uses HTTP (plaintext), browser may warn "not secure". Acceptable for LAN configuration scenarios, but for public internet access, must use HTTPS (requires TLS certificate, much more complex).

Note 4: Phone Browser Auto-Redirect

Some phone browsers auto-redirect [192.168.4.1](#) to [https://](#), causing access failure. Solutions:

- Manually enter full address [http://192.168.4.1](#)
- Or use a browser that doesn't force HTTPS redirect (like Firefox)

Note 5: `server.handleClient()` Must Be Called in loop

[Back to Elegoo](#)

Web server doesn't automatically process requests, must call `server.handleClient()` repeatedly in `loop()`. If a state handler takes too long, webpage response will be slow. This tutorial calls it in `handleAPFallback()`, only responding to web requests in AP mode.

Note 6: State Transitions Must Clean Up Resources

When leaving `AP_FALLBACK`, must call `stopAPFallback()` to stop web server and disconnect AP. Otherwise, after switching to STA mode, AP still running, will cause WiFi mode confusion.

Note 7: `ESP.restart()` Delay

`ESP.restart()` doesn't take effect immediately, waits a few hundred milliseconds. The `delay(500)` in code is to let HTTP response be sent first, avoiding user browser errors.

Note 8: First Use vs Subsequent Use

- **First time:** NVS empty → Direct AP portal → User configures → Save and reboot → STA connection
- **Subsequent:** NVS has credentials → Direct STA → Only enters AP portal on failure

This is "seamless upgrade" design: User plugs in at home, if new device auto-guides configuration; if already configured, directly connects to WiFi.

← AP mode

NTP Time→